

Project Description

Title :

Autoencoder Implementation & Regularization Study on MNIST

Objective :

Implement a basic autoencoder and a denoising autoencoder on the MNIST dataset using PyTorch. Compare how regularization techniques (L2 regularization, dropout, and early stopping) affect reconstruction error and visual output quality.

Project Overview :

The interns will:

- Implement a Basic Autoencoder in PyTorch on MNIST.
- Implement a Denoising Autoencoder with artificial noise injection.
- Apply L2 regularization, dropout, and early stopping.
- Compare reconstruction quality with a PCA baseline.
- Analyse the effect of different latent space sizes (16, 32, 64).

This project introduces interns to deep learning model design, regularization strategies, and visual evaluation of generative/reconstructive models.

Expected Features

Core Features (Mandatory)

1. Basic Autoencoder

Implement:

- Encoder: Fully-connected layers compressing 784 -> latent dim
- Decoder: Mirror architecture reconstructing latent -> 784
- Loss: Mean Squared Error (MSE) or Binary Cross-Entropy
- Optimizer: Adam

2. Denoising Autoencoder

Implement:

- Corrupt input images with Gaussian or salt-and-pepper noise
- Train model to reconstruct clean images from noisy input
- Compare reconstruction quality vs. basic autoencoder

3. Regularization Techniques

Apply and compare:

- L2 Regularization (weight_decay in optimizer)
- Dropout layers in encoder/decoder
- Early Stopping based on validation loss plateau

4. Key Analysis / Evaluation

Answer during/after implementation:

- Which model achieves better reconstruction: Basic AE, Denoising AE, or PCA?
- How does each regularization technique affect training loss and visual quality?
- What is the effect of latent space size (16 vs 32 vs 64) on reconstruction?

5. Responsive Visualisation

- Plot original vs. reconstructed images side-by-side
- Plot training and validation loss curves
- Display reconstruction MSE table across all models and latent sizes Search and Filter

Bonus Features (Advanced)

1. Sparse Autoencoder

- Add KL-divergence sparsity penalty
- Compare with standard L2-regularised autoencoder

2. Variational Autoencoder (VAE)

- Reparameterisation trick with mean and log-variance
- ELBO loss (reconstruction + KL divergence)
- Sample new digits from latent space

3. Anomaly Detection Demo

- Train only on digits 0–8; test on digit 9
- Use high reconstruction error as anomaly signal

4. Convolutional Autoencoder

- Replace FC layers with Conv2d / ConvTranspose2d
- Compare visual quality vs. FC autoencoder

5. Latent Space Visualisation

- t-SNE or UMAP plot of latent vectors coloured by digit class
- Observe clustering by class in 2D projection

Deliverables

At the end of **1 week** interns should submit:

Mandatory

- Working Jupyter Notebook / Python script
- GitHub Repository with clear commit history
- README with setup instructions and results summary
- Comparison table: MSE for all models × latent sizes
- requirements.txt

Suggested Tech Stack (Use According to Your Skills)

Area	Technology
Deep Learning Framework	PyTorch / TensorFlow
Baseline	scikit-learn PCA
Data Handling	torchvision.datasets.MNIST
Visualisation	matplotlib / seaborn
Notebook	Jupyter Notebook / Google Colab
Version Control	Git / GitHub

Dataset / Test Data

MNIST Handwritten Digits

- 60,000 training images, 10,000 test images
- 28×28 grayscale images of digits 0–9
- Available via: `torchvision.datasets.MNIST` or `tensorflow.keras.datasets.mnist`

Note :

- The references and steps in this document are provided only to help you understand the requirements. You can use any resources, tools, or approach that works best for you.
- These references are shared to save your research time and give you a starting point. Our main expectation is a fully working application that meets all requirements using your own development process and logic.